PipeDoctor: Project Technical Sheet

Oct 8, 2026 · @Dyman

PipeDoctor is an AI agent that acts as an on-call data engineer: when a Spark, Airflow or dbt pipeline fails on Azure Databricks, it diagnoses the cause from logs, lineage and table history, then opens a tested fix as a pull request that a human approves.

The problem

Pipeline failures are routine, repetitive and slow to fix, and the work falls on the few engineers who know the platform.

- **A small set of causes repeats.** Schema drift, bad values, late or missing files, skewed joins, out-of-memory errors, non-idempotent retries, expired credentials and broken dbt references account for most day-to-day breakage (hypothesis to validate in the interviews, see the management layer).
- **Diagnosis is manual.** The on-call engineer opens the orchestrator, reads logs, checks the table history and upstream jobs, guesses, patches, tests and writes the pull request by hand.
- **Alerts say that, not why.** Orchestrators report a failed task; they do not say which upstream change caused it or what a safe fix looks like.
- **Generic coding assistants lack context.** They cannot read run logs, Delta history or lineage, and nothing measures whether their fix is right or safe.

The gap this project targets is not "an LLM that reads logs". It is an agent with controlled access to run context, a validated fix, a human gate, and a benchmark that shows how often it is right.

The solution

PipeDoctor closes the loop from failure alert to reviewed fix, and measures every step.

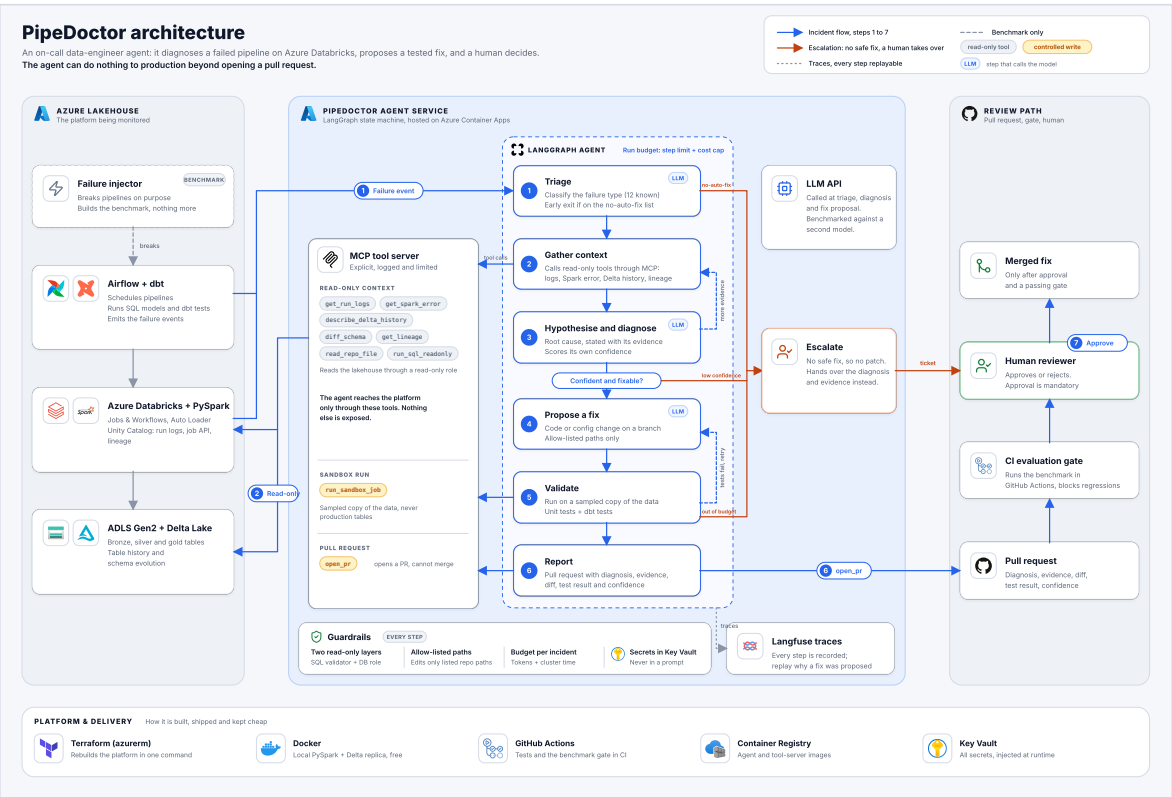
1. **Detect.** A failed Databricks job, Airflow task or dbt run sends a failure event.
2. **Gather context.** The agent calls read-only tools for run logs, the Spark error, Delta table history, a schema diff, lineage and the recent commits.
3. **Diagnose.** It classifies the failure type and states the root cause with the evidence it used.

4. **Propose.** It writes a fix on a separate branch: a code change, a configuration change, or an escalation when no safe fix exists.
5. **Validate.** The fix runs in a sandbox on a sampled copy of the data, with unit tests and dbt tests.
6. **Report.** It opens a pull request containing the diagnosis, evidence, diff, test result and a confidence level.
7. **Approve.** A human reviews; the CI evaluation gate runs; only then does the change merge.

In scope: diagnosis and fix proposals for about 12 injected failure types on one lakehouse platform on Azure; a benchmark with ground truth; the management documents.

Out of scope: merging to production without a human, modifying source data, fixing business-logic errors, and any claim about a real company's incidents.

Architecture



architecture · lakehouse, agent state machine, review path (full size on page 10)

The agent sees the lakehouse only through a read-only tool server, tries its fix in a sandbox, and can do nothing to production beyond opening a pull request; every step is traced.

Failure catalogue

The benchmark injects 12 failure types, each with a known root cause and a ground-truth fix. Two of them must end in escalation, not a fix, to test that the agent knows its limits.

#	Failure	Where it appears	Signal the agent sees	Expected outcome
1	Schema drift (new or renamed column)	Auto Loader ingest	Schema-mismatch error, schema diff	Code fix: column mapping or schema evolution
2	Type change (string to integer)	Delta write	Cast error, schema diff	Code fix: explicit cast
3	Null or invalid values	Delta constraint	Constraint-violation error, sample rows	Code fix: quarantine or filter step
4	Late or missing upstream file	Sensor or Auto Loader	Timeout, empty batch, file listing	Escalate: upstream owner, adjust retry policy
5	Skewed join (hot key)	Spark join	One straggler task, spill in the Spark UI	Config or code fix: skew handling, broadcast
6	Out-of-memory, executor lost	Spark job	OOM in executor log	Config fix: partitioning, memory settings
7	Small-file explosion	Delta table	Many tiny files, slow reads	Config fix: compaction, write settings
8	Duplicate rows after a retry	Delta append	Duplicate count, run history	Code fix: idempotent MERGE
9	Concurrent write conflict	Delta table	Concurrent-modification exception	Config fix: retry, partition the writes
10	Broken dbt reference	dbt model	Compilation error, missing ref	Code fix: correct the ref

#	Failure	Where it appears	Signal the agent sees	Expected outcome
11	Column removed upstream	dbt or Spark SQL	Column-not-found error, lineage	Code fix: update the select
12	Expired credential or denied permission	Key Vault or access control	Authorisation error	Escalate: never fix automatically

Each scenario ships as a small repository change plus a data change, so the failure is reproducible and the fix can be checked against the ground truth.

Agent design and guardrails

The agent is a LangGraph state machine that reaches the platform only through an MCP tool server, so every capability is explicit, logged and limited.

Flow: triage, gather context, hypothesise, propose a fix, validate in the sandbox, report. A step limit and a cost cap bound every run.

MCP tools (all read-only except the last two):

- `get_run_logs`, `get_spark_error`: logs and error text for a failed run.
- `describe_delta_history`, `diff_schema`: table versions and schema changes.
- `get_lineage`: upstream and downstream tables for the failing job.
- `read_repo_file`: files from the pipeline repository.
- `run_sql_readonly`: queries through a read-only role.
- `run_sandbox_job`: runs the proposed fix on a sampled copy, never on production tables.
- `open_pr`: opens a pull request on a branch; it cannot merge.

Guardrails:

- Two independent read-only layers: a SQL validator and a read-only database role.
- The agent may edit only an allow-listed set of paths in the pipeline repository.
- Secrets stay in Key Vault and never enter a prompt.
- A fixed budget per incident for tokens and cluster time; the run stops when it is spent.
- The agent must escalate when its confidence is low or the failure type is on the no-auto-fix list.

- Every step is traced, so a reviewer can replay why a fix was proposed.
- Human approval is mandatory; nothing reaches production without it.

Tools and stack

The stack follows the PFE offers I read: Python in every one, cloud in most (GCP in 4, Azure in 3, AWS in 1), plus Docker, CI/CD and Terraform. Development runs locally and the demo runs on Azure to keep cost low.

Layer	Tool	Role in the project
Cloud	Azure: ADLS Gen2, Key Vault, Container Apps, Container Registry	Storage for the lakehouse, secrets, hosting of the agent and tool server
Compute	Azure Databricks: Jobs and Workflows, Unity Catalog	Runs the Spark pipelines; provides run logs, job API and lineage
Processing	PySpark, Delta Lake, Auto Loader	Ingestion and transformation on a large real dataset; table history and schema evolution
Orchestration	Airflow and Databricks Workflows	Schedules pipelines and emits the failure events
Transformation	dbt	SQL models and tests, one of the failing layers
Agent	LangGraph and an MCP tool server	The decision loop and its controlled access to the platform
Model	LLM through an API, compared with a second model	Diagnosis and fix proposals; the choice is itself benchmarked
Tracing and lineage	Langfuse or LangSmith, OpenLineage	Replayable traces of each incident, lineage as agent input
Infrastructure as code	Terraform (azurerm)	Rebuilds the whole platform from one command

Layer	Tool	Role in the project
CI/CD	GitHub Actions	Tests, and a gate that runs the benchmark and blocks regressions
Containers	Docker	Local replica of the platform (PySpark and Delta) for free development
Evaluation	Python, pytest, bootstrap statistics	Benchmark runner and confidence intervals

Evaluation

The claim "it works" is replaced by a measured one: a benchmark of 50 to 60 injected scenarios with ground truth, scored against two baselines.

Metric	Definition
Diagnosis accuracy	Share of scenarios where the failure type and root cause match the ground truth
Fix validity	Share of proposed fixes that pass the sandbox run and the tests
Fix correctness	Share of valid fixes equivalent to the ground-truth change
Safe escalation rate	Share of no-auto-fix scenarios where the agent escalates instead of patching
Unsafe action rate	Actions outside the allow-list or read-only rules; the target is zero
Cost per incident	Tokens and cluster time per scenario, in dollars
Time to pull request	Wall-clock time from failure event to opened PR

Baselines: (a) the same model given only the error log and no tools; (b) a rule-based runbook that maps error strings to fixes. **Ablations:** remove lineage, remove Delta history, remove sandbox validation, one at a time, to show which tool earns its place.

Statistics: each scenario runs several times because model output varies; results are reported as a mean with a bootstrap 95% confidence interval. Prompts are tuned on a development split and scored once on a held-out split.

Known limit: injected failures are not real incidents. The scenarios are therefore drawn from what the interviewed data engineers describe, and the report states this limit.

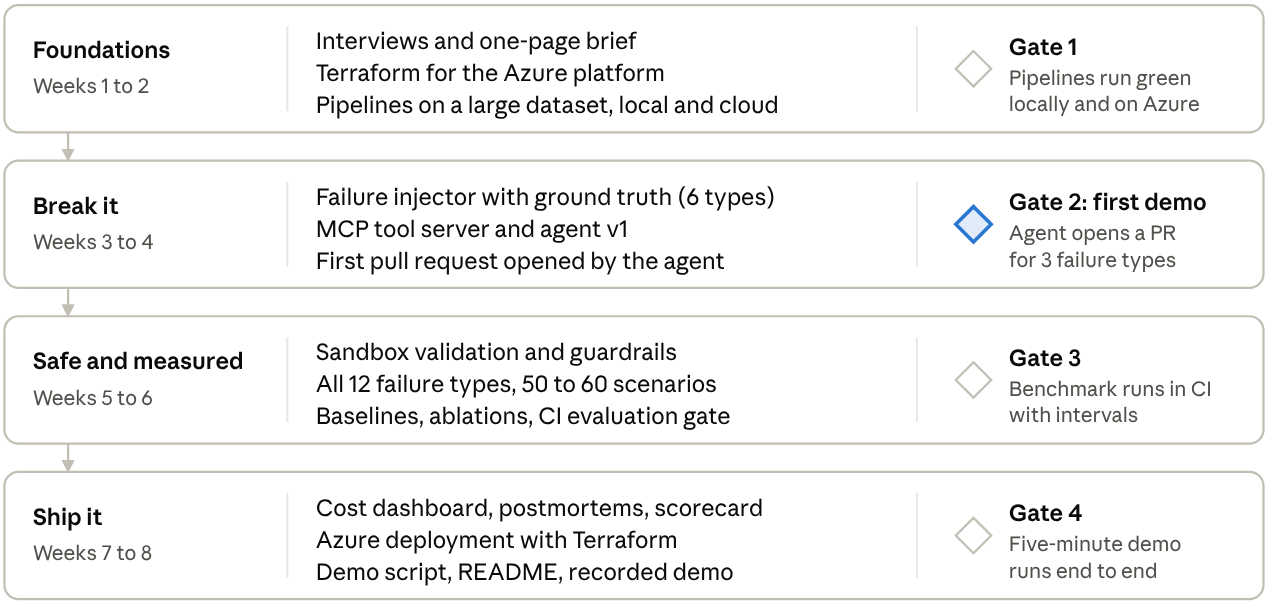
Management layer

The project is run like a small product, and the documents below are part of the deliverable, not an afterthought.

Artifact	What it shows	Week
User interviews	3 to 5 data engineers describe their worst on-call incidents; the failure catalogue is adjusted to match	1
One-page brief	Problem, users, goals, KPIs and service-level targets, scope and non-goals	1
Decision log	One short record per major choice (cloud, model, orchestrator), with the alternatives rejected	1 to 8
Risk register	Risks with owner, probability, impact and mitigation, reviewed each phase gate	1 to 8
Cost dashboard	Cloud and model spend per incident and per week against a fixed budget	7
Postmortems	One generated per benchmark failure the agent got wrong, with the lesson	6 to 7
Reliability scorecard	The evaluation metrics on one page, per failure type	7
Demo script	A five-minute walk-through from failure to merged fix, rehearsed	8

Roadmap

Eight weeks, four phases, each ending at a gate



Highlighted: the first demoable result, at the end of week 4.

roadmap · 4 phases, 4 gates, 8 weeks

Week 4 gives the first demoable result, so the project stays presentable even if later phases slip. Each gate is a go or no-go review recorded in the decision log.

Risks and mitigations

Risk	Why it matters	Mitigation
Injected failures differ from real incidents	Results may not transfer to production	Build scenarios from the interviews; state the limit in the report
Scope creep	12 failure types, a platform and a management layer can swallow the time	Fixed order: 6 types first, then the rest; each phase ends at a gate
Model output varies run to run	One lucky run can mislead	Repeat runs and report confidence intervals

Risk	Why it matters	Mitigation
Cloud cost	Databricks clusters bill while idle	Develop locally in Docker; auto-terminate clusters; budget alert and cost cap
Free-tier limits	Lineage or job features may be missing on a free or student plan	Check before building on them; fall back to table history and parsed logs
The agent takes an unsafe action	Trust in the project collapses	Read-only layers, allow-listed paths, mandatory approval, an unsafe-action metric
Time competes with PFE applications	The project stays unfinished	Eight weeks with a demoable result at the end of week 4

Deliverables and points to verify

Deliverables

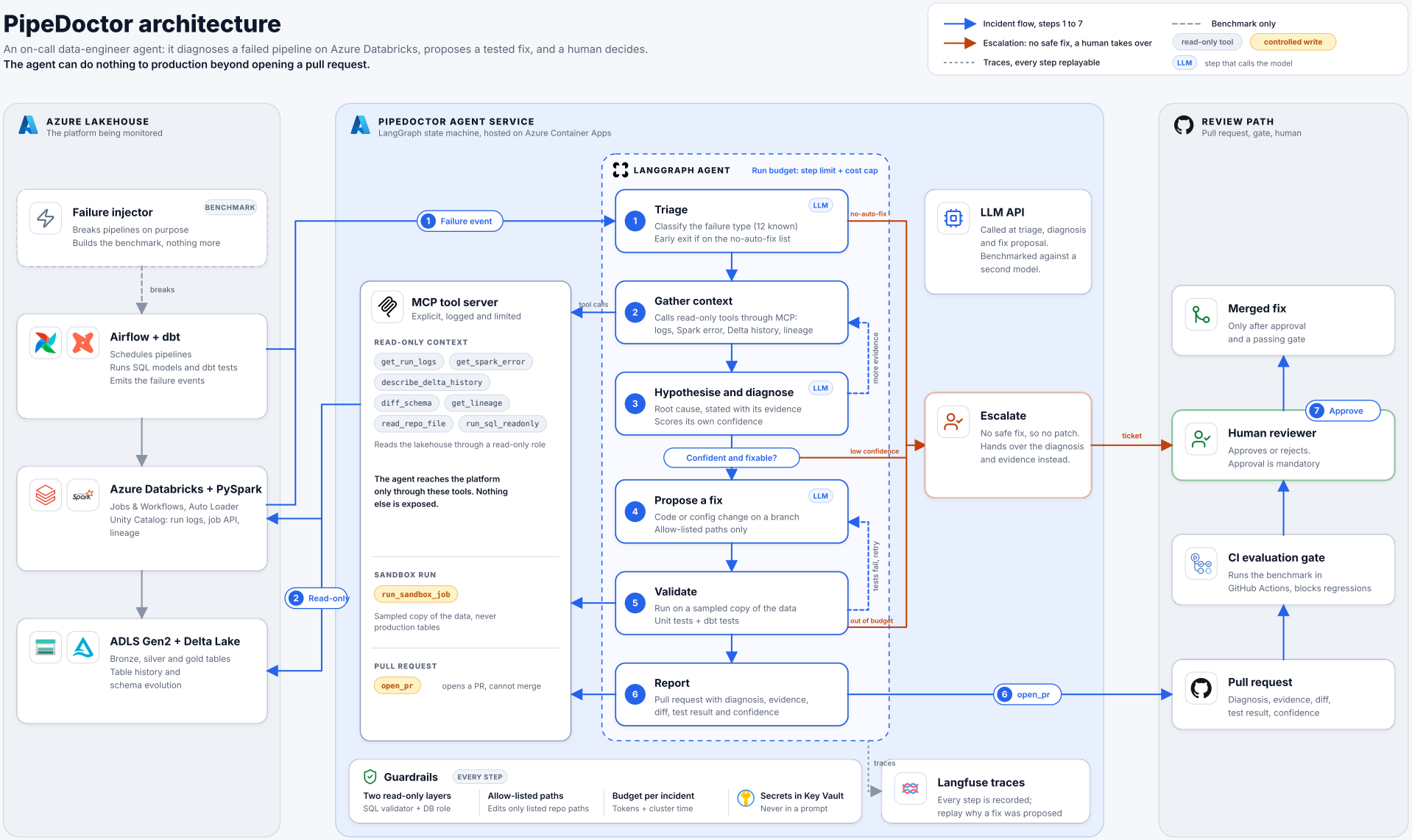
- A GitHub repository with the Terraform, the platform pipelines, the MCP server, the agent and the benchmark runner.
- A benchmark of 50 to 60 failure scenarios with ground truth.
- An evaluation report with baselines, ablations and confidence intervals.
- The management pack: brief, decision log, risk register, cost dashboard, scorecard.
- A recorded demo, a README with the architecture, and two lines for the CV.

To verify before starting (nothing in this sheet was checked against live documentation)

- The current Azure student offer and its credit limits.
- What Databricks Free Edition allows: the Jobs API, cluster options, and whether Unity Catalog lineage is available.
- The large real dataset that justifies Spark; not chosen yet, and its licence must allow redistribution of the scenarios.
- The language model and its cost per incident, which the benchmark will compare against a second model.
- Whether to keep Airflow, or use Databricks Workflows only, if the two overlap too much.

PipeDoctor architecture

An on-call data-engineer agent: it diagnoses a failed pipeline on Azure Databricks, proposes a tested fix, and a human decides.
The agent can do nothing to production beyond opening a pull request.



PLATFORM & DELIVERY How it is built, shipped and kept cheap

- **Terraform (azurerm)** — Rebuilds the platform in one command
- **Docker** — Local PySpark + Delta replica, free
- **GitHub Actions** — Tests and the benchmark gate in CI
- **Container Registry** — Agent and tool-server images
- **Key Vault** — All secrets, injected at runtime